

Ordenamiento de Arreglos

Asignatura: Fundamentos de la Programación

Docente: Ing. Raúl Fernández Fajardo



Ordenamiento de Arreglos

Asignatura: Fundamentos de la Programación

Docente: Ing. Raúl Fernández Fajardo

Objetivo General

Comprender la importancia del ordenamiento de arreglos en la organización y manipulación eficiente de datos.

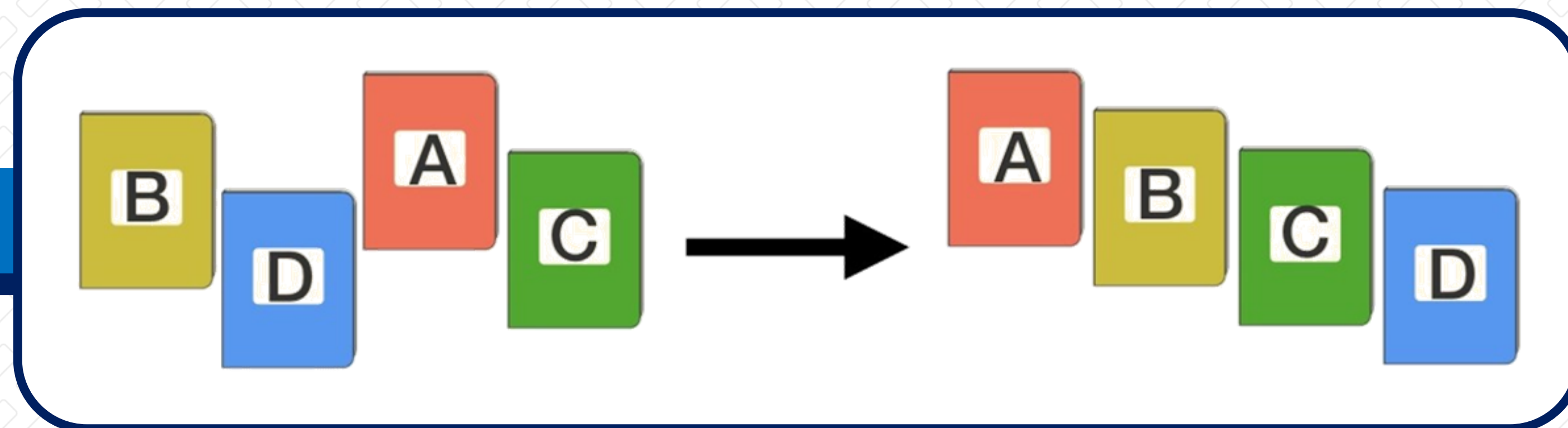
Objetivos Específicos

- Explicar el concepto de ordenamiento de datos y su relevancia en diferentes aplicaciones del mundo real.
- Analizar la función sort y otros métodos.

El ordenamiento de datos es una de las operaciones más comunes y esenciales en programación. **Permite organizar la información de forma lógica**, facilitando búsquedas, comparaciones, análisis y presentación.

Ejemplos cotidianos del ordenamiento:

-  Buscar un nombre en una **lista de alumnos ordenada alfabéticamente**.
-  Ver productos ordenados por **precio** en una tienda en línea.
-  Organizar transacciones bancarias por **fecha o monto**.
-  Ordenar los datos antes de aplicar algoritmos de análisis o inteligencia artificial.



“El orden en los datos es el primer paso para tomar decisiones inteligentes.”

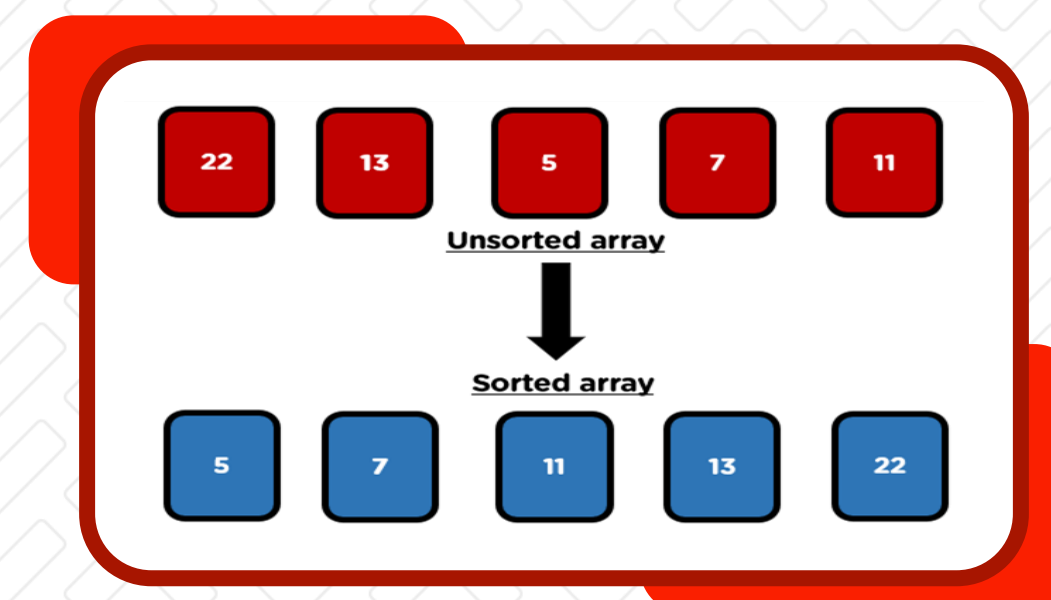
Ordenamiento y la función `sort()` en C++

El ordenamiento de datos es un problema intrigante que ha atraído algunos de los esfuerzos de investigación más intensos en el campo de las ciencias computacionales.

En C++, **la función `sort()` es una herramienta** poderosa incluida en la librería estándar STL, diseñada para ordenar arreglos y estructuras de datos de forma rápida y eficiente, gracias al uso de algoritmos avanzados como IntroSort, una combinación optimizada de QuickSort, HeapSort e Insertion Sort.

Por defecto, esta función **organiza los elementos en orden ascendente**, lo cual la convierte en una solución práctica, robusta y altamente utilizada en el desarrollo de software moderno.

"Ordenar con `sort()` no solo pone en fila los datos, también organiza tus ideas para resolver problemas con eficiencia.



Ejemplo de la función “sort”

main.cpp

```
1  #include <iostream>    // Para usar cout
2  #include <algorithm>    // Para usar sort()
3
4  using namespace std;
5
6  int main() {
7      // Arreglo de números desordenados
8      int numeros[] = {7, 2, 5, 1, 9};
9
10     // Tamaño del arreglo
11     int tam = sizeof(numeros) / sizeof(numeros[0]);
12
13     // Ordenar el arreglo de menor a mayor
14     sort(numeros, numeros + tam);
15
16     // Mostrar el arreglo ordenado
17     cout << "Arreglo ordenado: ";
18     for(int i = 0; i < tam; i++) {
19         cout << numeros[i] << " ";
20     }
21
22     return 0;
23 }
```

Output

Arreglo ordenado: 1 2 5 7 9

`int tam = sizeof(numeros) / sizeof(numeros[0]);`

✓ ¿Qué hace esta línea?

Calcula el número de elementos que hay en el arreglo numeros.

🔍 Paso a paso:

1. `sizeof(numeros)`

- Esto devuelve el tamaño total en bytes del arreglo completo.
- Ejemplo: Si el arreglo es `int numeros[] = {7, 2, 5, 1, 9};` Tiene 5 elementos, y cada `int` ocupa normalmente 4 bytes, entonces:

`sizeof(numeros) = 5 * 4 = 20 bytes`

2. `sizeof(numeros[0])`

- Esto devuelve el tamaño en bytes del primer elemento del arreglo.
- En este caso, `numeros[0]` es un `int`, que ocupa 4 bytes.

3. División

`int tam = 20 / 4; // = 5`

Así se obtiene el número total de elementos en el arreglo.

♦ `sort(numeros, numeros + tam);`

👉 Le indica a C++ que ordene desde la primera posición (`numeros`) hasta la última (`numeros + tam`).

♦ El resultado será: 1 2 5 7 9

💡 ¿Por qué es útil?

Porque en arreglos estáticos no hay una función como `.length()` o `.size()` como en otros lenguajes. Esta es una forma segura y automática de saber cuántos elementos tiene el arreglo

Además de utilizar la función sort, también podemos aplicar diferentes algoritmos de ordenamiento que permiten organizar datos de forma particular, basándose en un criterio específico, como el valor, el orden alfabético o una clave determinada..

Estos algoritmos:

- Reciben un arreglo o lista como entrada.
- Organizan los elementos según un orden definido (por ejemplo, de menor a mayor o por orden alfabético).



Características

- Los datos se pueden ordenar alfabéticamente o por valores numéricos.
- El orden puede ser tanto ascendente (menor a mayor) como descendente (mayor a menor).
- Son herramientas útiles para reorganizar información, facilitando el acceso, la búsqueda y la presentación de los datos de forma clara.

Algoritmos de ordenación

 **Tabla Comparativa: Algoritmos de Ordenamiento en C++**

Algoritmo	Uso común / Aplicación	Función / Idea principal	Librería (<code>#include</code>)
<code>sort()</code> (STL)	Ordenamiento rápido en programas reales	Usa IntroSort (QuickSort + HeapSort + Insertion Sort)	<code><algorithm></code>
Bubble Sort	Enseñanza básica, comprensión visual	Compara pares y "burbujea" el mayor al final	Implementación manual
Insertion Sort	Listas pequeñas o parcialmente ordenadas	Inserta cada elemento en su lugar dentro de una lista	Implementación manual
Selection Sort	Muy simple, pero poco eficiente	Encuentra el mínimo y lo coloca en su posición	Implementación manual
QuickSort	Ordenamiento eficiente en la mayoría de casos	Divide y conquista: particiona y ordena recursivamente	Implementación manual
MergeSort	Datos muy grandes, requiere más memoria	Divide en mitades, ordena y une	Implementación manual
HeapSort	Ordenamientos sin recursividad	Usa una estructura de heap (árbol binario) para ordenar	Implementación manual
Counting Sort	Números pequeños con valores repetidos	Cuenta ocurrencias y reconstruye el arreglo	Implementación manual
Radix Sort	Enteros largos o cadenas numéricas	Ordena por dígitos, de menor a mayor	Implementación manual

 Notas:

 En esta clase solo aprenderemos a usar la función `sort()`, pero es importante que **conozcas otros métodos de ordenamiento**, ya que se estudian con más profundidad en materias como **Estructuras de Datos**.

 **Te animo a investigar por tu cuenta** cómo funcionan los algoritmos como burbuja, selección o inserción, y si puedes, ¡intenta escribirlos en código!

 Algunos de esos algoritmos, como **QuickSort y MergeSort**, son los que realmente usa `sort()` por dentro, así que **entenderlos te ayudará a comprender cómo trabaja tu código** en el fondo.

"Implementación manual"

- ◆ **No viene incluido automáticamente** en una librería estándar de C++ (como `sort()` que sí está en `<algorithm>`).
- ◆ Tú como programador debes **escribir el código completo del algoritmo** paso a paso, utilizando bucles, condiciones y lógica propia.

Ordenamiento de Arreglos

Asignatura: Fundamentos de la Programación

Docente: Ing. Raúl Fernández Fajardo

Ordenamiento de Arreglos

Asignatura: Fundamentos de la Programación

Docente: Ing. Raúl Fernández Fajardo

